

Assignment 2

Student: Francesco Gnan; Student's email: gnanf@usi.ch

November 14, 2023

1 Image classification using CNNs [90/100]

In this Assignment we want to perform image classification using convolutional neural networks (CNNs). In particular, we use the CIFAR-10 dataset which consists of 60000 (50000 for the training and 10000 for the test) 32x32 color images across 10 different classes, with each class containing 6000 images. These classes include common objects such as airplanes, automobiles, cats, and dogs. Our goal is to correctly classify these images.

1.1 Data (20 pts)

1. First of all we load the dataset using `torchvision.datasets.CIFAR10`. We initially load the data into two variables: 50000 in `trainset` and 10000 in `testset`. This split is made by setting the parameter `train` to `True/False` as done in lines. Then we inspect the dataset. We want to observe at least one image per class. We do it looping over the images and creating a dictionary where labels are the keys and images are the corresponding values. This is done in lines 62-66 of file `Gnan.Francesco.py`. Afterwards in lines 68-74 we show these images and their labels using the provided function `imshow()`. These examples are reported in Figure 1.

We observe that these images have different frames and the objects have different positions within the image. Our network is asked to be able to generalize well on these differences, i.e. it has to be able to recognize rotational and translational symmetry. At this point we want to create a histogram of the distribution of the images of the training and test set. To get this result we use the functions `create_histogram()` (lines) and `plot_histograms()` (lines 79-103). They produce a python dictionary, whose keys are the labels and whose values are the images, and



Figure 1

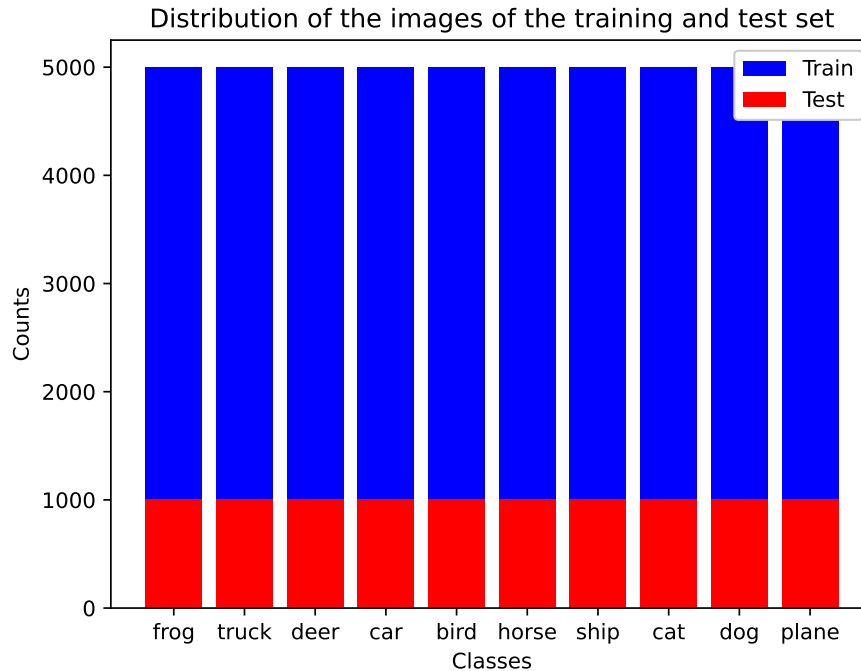


Figure 2

they use it to plot the requested histogram. We observe in Figure ?? that the training dataset is five times larger than the test one and that each class contains the same number of images.

2. The CIFAR-10 dataset contains 60000 32x32 color images in 10 different classes. To work in PyTorch we want each entry to be a `torch.Tensor` of shape (3, 32, 32). In lines 117-122 we check the datatypes before the transformation into tensors. Before the transformation, each element of the dataset is a tuple. The tuple is composed by a `PIL.Image.Image`, which is the image itself, and a `int` value, which is the label information. Then from line 127 to line 129 we transform the data into tensors. Afterwards in lines 134-136 we check the dimension of an image as a `torch.Tensor` object: we observe that after the transformation the dimension of each image is `torch.Size([3, 32, 32])`, where 3 is the number of color channels and 32x32 is the image size in pixels.
3. In order to work with features having mean 0 and standard deviation equal to 1, we use `torchvision.transforms.Normalize` as done in lines 146-148.
4. We want to create a third dataset for validation. We get it randomly splitting the training set and taking the 20% of the data from it. We do this operation in lines 153-154¹.

¹We used the following reference: <https://discuss.pytorch.org/t/torch-utils-data-dataset->

1.2 Model (10 pts)

From line 174 to line 227 we define our model. We shuffle and batch the training dataset using `DataLoader` because we want to expose our model to many examples in different orders during training. Then we set `shuffle=True` for the training set and `shuffle=False` for the validation and the test ones. This ensures that the model is exposed to diverse and random samples during training while maintaining a consistent order for evaluation on the validation and test datasets. We evaluate the model's performances on the validation and test datasets without updating the model parameters. This is why we don't need to shuffle the order of the examples.

To construct our model we used the following formula, that defines the output size given an input size, a kernel size and padding and stride informations:

$$\text{out} = \frac{\text{in} - \text{kernel} + 2 \cdot \text{padding}}{\text{stride}} + 1$$

The architecture we defined is as follows:

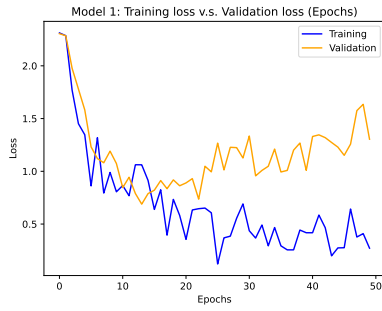
```
conv1: in_ch=3, out_ch=32, kernel=3, stride=1, padding=0
ReLU function
conv2: in_ch=32, out_ch=32, kernel=3, stride=1, padding=0
ReLU function
MaxPool1: size=2x2, stride=2, padding=0
-----
conv3: in_ch=32, out_ch=46, kernel=2, stride=1, padding=0
ReLU function
conv4: in_ch=46, out_ch=46, kernel=2, stride=1, padding=0
ReLU function
MaxPool2: size=2x2, stride=2, padding=0
-----
conv5: in_ch=46, out_ch=64, kernel=2, stride=1, padding=0
ReLU function
conv6: in_ch=64, out_ch=72, kernel=2, stride=1, padding=0
ReLU function
MaxPool3: size=2x2, stride=2, padding=0
-----
fc1: linear layer, input=2x2x72, output=120
ReLU function
fc2: linear layer, input=120, output=20
ReLU function
fc3: linear layer, input=20, output=10
```

We choose not to put a pooling layer after each convolutional layer because we did not introduce a padding and the input image has a small size (32x32). Reducing the number of pooling layers in fact we can add more convolutional layers.

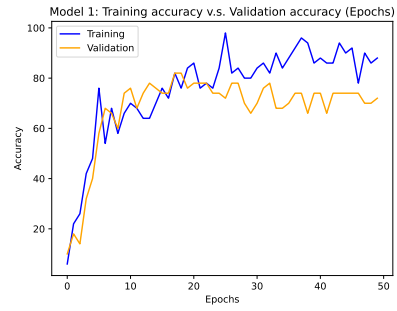
random-split/32209/4

1.3 Training (60 pts)

1. From line 231 to line 309 we train our model `ConvNet()`. Initially we choose the Cross Entropy as loss function and the Stochastic Gradient Descent as optimizer. The optimal hyperparameters we tuned during the training are: learning rate = 0.01, momentum = 0.9, number of epochs = 50. In lines 241-245 we transfer our model on a GPU in order to make our training faster. Finally in lines 249-309 we perform the training itself. we decided to save the training and validation loss and accuracy after each iteration and after each epochs. Since the training set is bigger than the validation one, in lines 280-284 we use a `if` condition in order to get two vectors of the same length, `training_loss` and `validation_loss`. Given a certain number of images, we define the accuracy as the number of correct predictions over the number of these images.
2. Even if the model overfits the validation data, it achieves an accuracy of 70.95% on the test dataset. In lines 330-345 we want to see the model acting on the test data: for each image within a batch we compare the true label with the predicted one (we use `F.softmax` to give a probability to each class and we take the one with highest probability using `torch.argmax`. For example we observe that our first model is able to recognize 37/50 classes.
3. In lines 349-352 we save the parameters of our trained model. It is possible to load these parameters with the command `net.load_state_dict` and use the net to perform new predictions or to continue its training.
4. We then plot the loss and the accuracy for this training phase. Figure 3 shows the training and validation loss and accuracy as functions of the number of the epochs, while Figure 4 as functions of each iteration. We decided to plot the latter in order to observe the big oscillations in the values. We did not get such a behavior when we trained our models in the first Assignment. This means that the space of model's parameters associated with the fitting task of Assignment 1 and the space associated to the classification task of the present Assignment are different. As far as image classification is concerned the number of parameters we set in our model is much larger than the number we set for linear regression. This explains why during the training phase is more difficult to find a global minimum for the loss and the model get trapped into many local minima. However as long as the number of epochs increases, the global trend of the loss is decreasing as expected. We observe an overfitting since the validation loss tends to become higher than the training one as far as the training process proceeds. This means that our model is learning well the training data but it is not completely able to generalize on unseen data. We expect that reducing the number of the parameters of the model, or introducing some training techniques, such as the dropout, the overfitting will be reduced.
5. We now enhance our model in order to achieve an higher accuracy on test data. In particular, in lines 417-480 we define the following architecture:

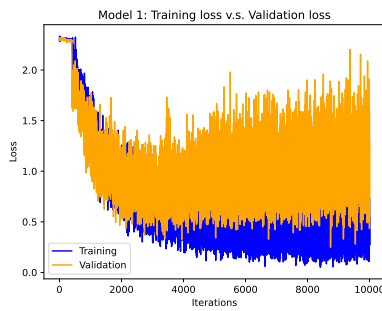


(a) Loss

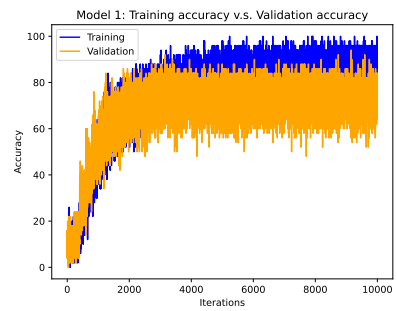


(b) Accuracy

Figure 3: Model 1, loss and accuracy as functions of the epochs



(a) Loss



(b) Accuracy

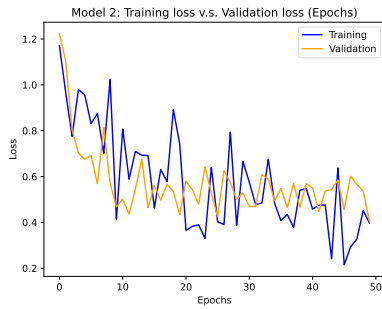
Figure 4: Model 1, loss and accuracy as functions of the iterations

```

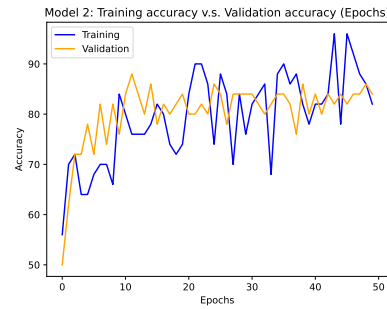
conv1: in_ch=3, out_ch=32, kernel=3, stride=1, padding=0
GeLU function
conv2: in_ch=32, out_ch=48, kernel=3, stride=1, padding=0
Batch Normalization
GeLU function
MaxPool1: size=2x2, stride=2, padding=0
Dropout(0.2)
-----
conv3: in_ch=48, out_ch=64, kernel=2, stride=1, padding=0
GeLU function
conv4: in_ch=64, out_ch=64, kernel=2, stride=1, padding=0
Batch Normalization
GeLU function
MaxPool2: size=2x2, stride=2, padding=0
Dropout(0.3)
-----
conv5: in_ch=64, out_ch=120, kernel=2, stride=1, padding=0
GeLU function
conv6: in_ch=120, out_ch=120, kernel=2, stride=1, padding=0
Batch Normalization
GeLU function
MaxPool3: size=2x2, stride=2, padding=0
Dropout(0.5)
-----
fc1: linear layer, input=2x2x220, output=100
Batch Normalization
GeLU function
Dropout(0.2)
fc2: linear layer, input=100, output=60
Batch Normalization
GeLU function
fc3: linear layer, input=60, output=10

```

Comparing this model with the first one, we augmented the number of parameters. In particular, the convolutional layers now produce in general much more features maps. We choose a different activation function, which is known to work well with images: GeLu. Since in Figures 3 and 4 we observed overfitting, in this new model we introduce some techniques useful to prevent it. For example, after each block of convolutional layers we apply the so-called Batch Normalization. It normalize the input of the following layer by adjusting and scaling the activations. During training, the distribution of inputs to each layer can change (covariate shift); the normalization brings them to a similar scale. Moreover, at the end of each convolutional layer block we add a Dropout. It prevents overfitting by randomly setting to zero a fraction of the neurons in a layer during training. In other words it helps the network to generalize. We used Dropout and Batch Normalization once also between the linear layers. As we can see in Figures 5 and 6, the regularization techniques introduced eliminated the overfitting. This improved model

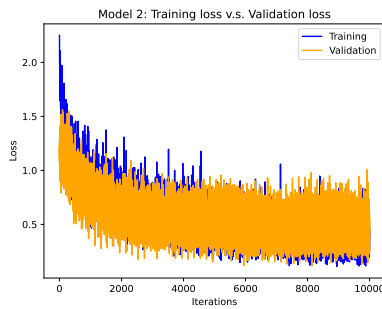


(a) Loss

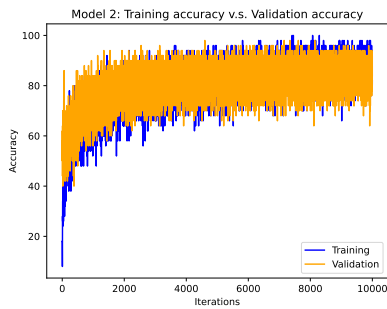


(b) Accuracy

Figure 5: Model 2, loss and accuracy as functions of the epochs



(a) Loss



(b) Accuracy

Figure 6: Model 2, loss and accuracy as functions of the iterations

reaches an accuracy of 83.89% on test data. For this training we changed the optimizer: we used Adam that is an adaptive method, i.e. it adapts the learning rates for each parameter individually. We used 50 epochs.

6. From line 582 to line 585 we save the parameters of the trained network as we did for model 1.

1.4 Bonus question* (5 pts)

We now want to train model 2 using 10 different seeds, in order to investigate how the test accuracy changes as far as the random seed is modified. In lines 654-721 we trained the model with 10 epochs. Table 1 shows the results. We observe that the test accuracies are slightly different. Therefore if we want to report a final result of the performance of our model, we should take the mean of these values and also provide a standard deviation. We can define as robust a model whose standard deviation is quite narrow (taking into account that it gets smaller as far as the number of accuracy values increases). Thus, if we want to compare two models, we should train them with the same set of seeds and provide for both of them a final average of the accuracies they achieve on the test set. The one with the best mean accuracy is to be considered preferable,

taking into account the width of the standard deviation.

Seed	Accuracy (%)
0	78.11
1	77.91
2	78.03
3	78.98
4	77.82
5	77.30
6	77.60
7	78.36
8	77.82
9	77.96

Table 1: Test accuracy for different seeds

Questions [5 points]

Residual Networks (ResNets) are a specific kind of architecture. They guarantee that, as far as the network complexity increases, the classes of functions that they can reach nested. Nested classes are more likely to get closer and closer to the 'true' function than non-nested ones. Moreover, ResNets can solve the vanishing gradient problem and thus are more easily trainable than standard convolutional neural networks. Its structure is as follows: they are organized in residual blocks. Let $\mathbf{x}$ be the input of one of these blocks and $f(\mathbf{x})$ the desired underlying mapping we want to obtain by learning. $f(\mathbf{x})$ is then given by $f(\mathbf{x}) = g(\mathbf{x}) + \mathbf{x}$, where $g(\mathbf{x})$ is the so-called residual mapping, i.e. the output of a certain set of layers and activation functions. The output of the residual block is $f(\mathbf{x})$, i.e. the sum of the output of the residual mapping and the input itself. In order to be added together they should have the same dimensions. With the term 'skip connections' we then refer to the transfer of a copy of the input directly before a certain block of layers and activation functions.